University: 8 May 1945 Guelma
Faculty: Mathematics, Computer Science and Matter Sciences
Department: Computer Science

Chapter 1

Introduction to Parallel Computing

- Course: High Performance Computing for Computer Science Engineers



  - 4th-year Computer Science Engineers "Artificial Intelligence"

Presented by: Dr. Rochdi Boudjehem

Academic Year: 2025/2026

Course Objectives

By the end of this chapter, students will be able to:

- Apply Amdahl's and Gustafson's laws to analyze parallel performance.
- Classify parallel architectures using Flynn's taxonomy.
- Differentiate between shared-memory, distributed-memory, and accelerated systems.
- Identify appropriate programming models for various parallel architectures.

1. Motivation

Why Parallel Computing?

Need for Parallel Computing

- Parallel computing has become essential as improvements in single-core performance have slowed down.
- Today’s applications in science, engineering, and industry require processing power from multiple cores and devices.
- This shift makes understanding parallel architectures and programming models crucial for modern computer scientists and engineers.

End of Dennard Scaling

Historical Context

- Mid-2000s: End of Dennard scaling
- Frequency scaling: Hit physical limits
- Power & Heat: Major constraints
- Single-thread performance: Has stopped increasing significantly (“plateaued”).

> *Modern CPUs no longer get much faster for a single core; improvements come from adding more cores and parallelism.*

Modern Reality

- Workload demands: Ever-larger datasets
- Multicore & Manycore: Standard architectures
- Parallel hardware: CPUs, GPUs, clusters, accelerators
- Solution: *Multiple operations simultaneously*

> *The primary goal is to speed up solutions or handle larger, more complex problems by executing multiple operations simultaneously.*

- Dennard Scaling

> *Dennard Scaling: As transistors get smaller, power density stays constant → power scales with area Ended in the mid-2000s due to leakage currents and heat dissipation limits*

Application Areas of Parallel Computing

Scientific Computing

- Computational Fluid Dynamics
- Numerical Weather Prediction
- Climate Models
- Molecular Dynamics

AI & Machine Learning

- Distributed Training
- Large-scale Inference
- Neural Architecture Search
- Hyperparameter Optimization

Bioinformatics

- Genomics Analytics
- Protein Folding
- Drug Discovery
- Phylogenetic Analysis

Computational Finance

- Risk Analysis
- Monte Carlo Simulations
- Algorithmic Trading
- Portfolio Optimization

2. Performance Basics

Metrics That Matter

Key Performance Metrics

Time-based Metrics

- Wall-clock time: Total elapsed time
- Latency: Time per operation
- Throughput: Operations per second

Computational Metrics

- FLOP/s: Floating-Point Operations per second
  - Giga (10^9), Tera (10^{12}), Peta (10^{15})
- Bandwidth: Data transfer rate (GB/s)

Parallel Performance

Speedup: $S(T) = \frac{T_1}{T_T}$

Efficiency: $E(T) = \frac{S(T)}{T}$

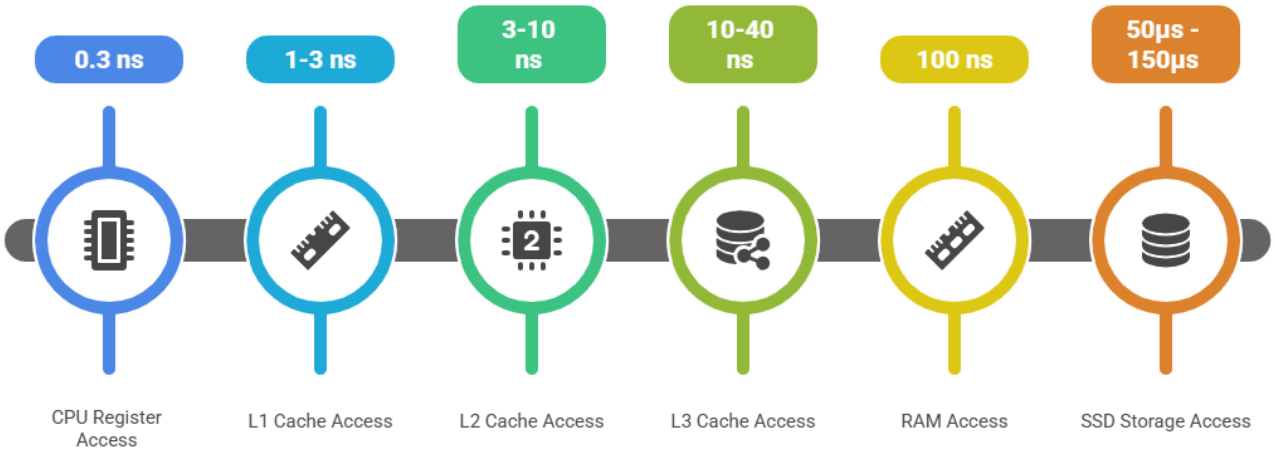
Where:

- T_1: Serial execution time
- T_T: Time with T workers

Scalability: Performance evolution with resources

> *Understanding these metrics is crucial for analyzing and optimizing parallel programs.*

Memory Hierarchy Impact



Performance Strategy

- Temporal Locality: Reuse data in faster levels
- Spatial Locality: Access nearby data together
- Cache Optimization: Minimize slow memory accesses

> *Minimizing accesses to slower memory levels is key for high performance.*

Types of Parallelism

See More

Data Parallelism

- Same operation on different data
- Distribute data across processors
- Example: Vector operations, image processing

Pipeline Parallelism

- Sequential stages processed in parallel
- Assembly line approach
- Example: Instruction pipelines, data processing

Task Parallelism

- Different tasks executed concurrently
- Independent or loosely coupled tasks
- Example: Producer-consumer, pipeline stages

Hybrid Approaches

- Combine multiple strategies
- Optimize for complex applications
- Example: MPI + OpenMP, GPU + CPU

Concurrent vs Parallel Execution

Concurrent Execution

- Multiple tasks handled at the same time
- Not necessarily simultaneously
- Uses time-slicing on single processor
- Creates illusion of parallelism

Key Techniques:

- Scheduling
- Time-slicing
- Context switching

Parallel Execution

- Task division into smaller sub-tasks
- True simultaneous execution
- Requires multiple processors
- Needs explicit coordination

Implementation:

- Multiple processors/cores
- Process coordination
- Shared resources management

Key Differences: Concurrency vs Parallelism

Concurrency

Appearance of Simultaneity

- Single processor with time-sharing
- Interleaved execution of tasks
- Context switching creates illusion
- Scheduling algorithms manage fairness
- Tasks appear to run at same time

💡 Like a juggler keeping multiple balls in the air

Parallelism

True Simultaneity

- Multiple processors working together
- Simultaneous execution of tasks
- No time-sharing required
- Coordination between processors
- Tasks actually run at same time

🚀 Like multiple workers building different parts simultaneously

Concurrent vs Parallel: Key Distinction

The Distinction

Concurrent: Tasks *appear* to run simultaneously (OS scheduling creates illusion) Parallel: Tasks *actually* run simultaneously (multiple execution units)

> *Operating systems use scheduling, time-slicing, and context switching to manage concurrent tasks on limited processors*

Concurrency Features

- Interleaving execution on single core
- Context switching overhead
- Scheduling algorithms manage fairness
- Appears simultaneous to user

Parallelism Features

- True simultaneous execution
- Multiple processing units required
- Coordination between processors
- Actually simultaneous execution

Example: Sequential vs Parallel Execution

Sequential Execution

0. Read input data
0. Preprocess data
0. Execute main computation
0. Postprocess results
0. Write output data

Characteristics:

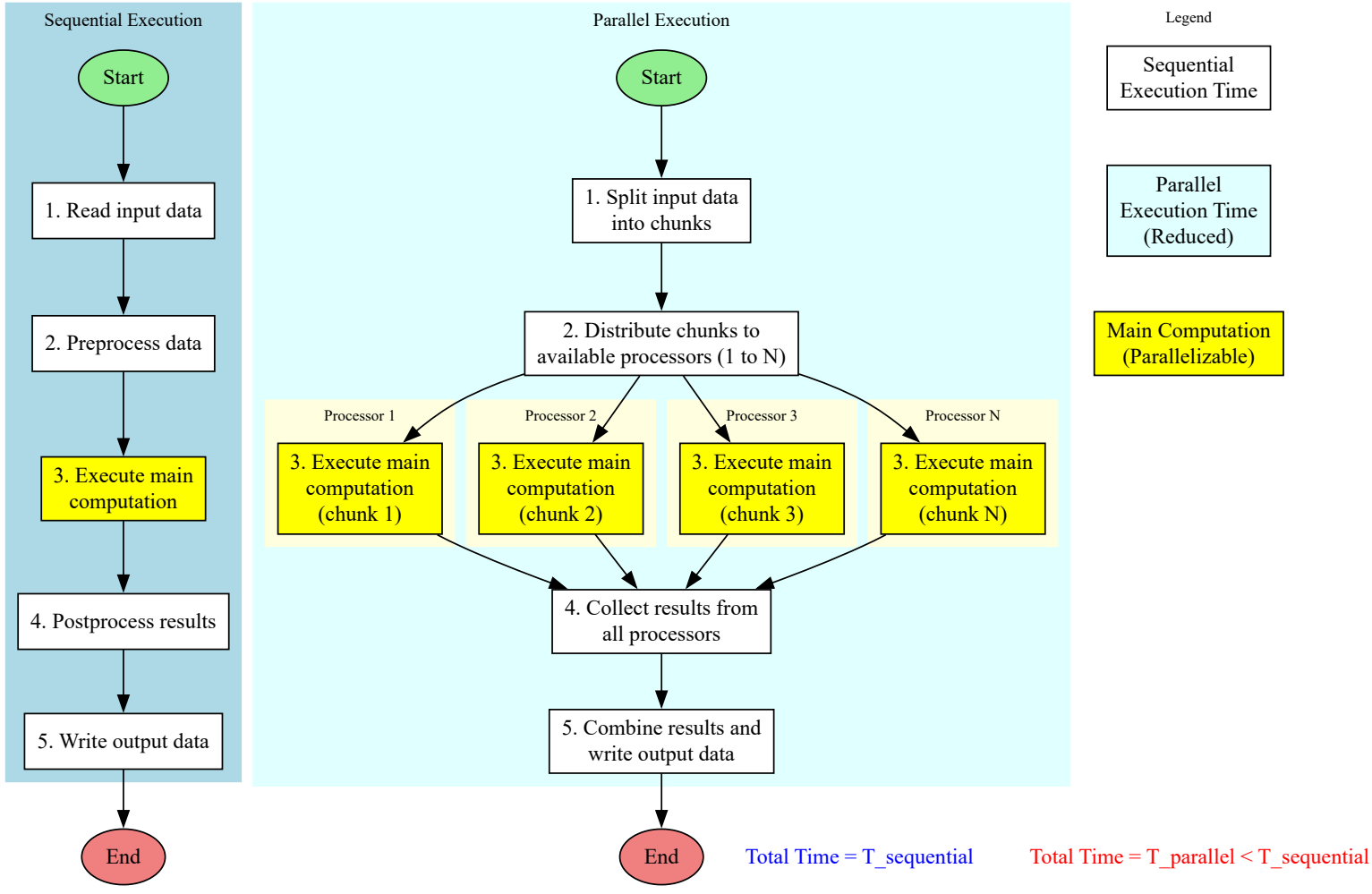
- Linear execution
- Single processor
- Total time = Sum of all steps

Parallel Execution

0. Split input data into chunks
0. Distribute chunks to processors
0. Each processor computes its chunk
0. Collect results from processors
0. Combine results and write output

Characteristics:

- Concurrent execution
- Multiple processors
- Total time < Sequential time


Sequential Execution
Start
1. Read input data
2. Preprocess data
3. Execute main computation
4. Postprocess results
5. Write output data
End
Parallel Execution
Start
1. Split input data into chunks
2. Distribute chunks to available processors (1 to N)
Processor 1
3. Execute main computation (chunk 1)
Processor 2
3. Execute main computation (chunk 2)
Processor 3
3. Execute main computation (chunk 3)
Processor N
3. Execute main computation (chunk N)
4. Collect results from all processors
5. Combine results and write output data
End
Legend
Sequential Execution Time
Parallel Execution Time (Reduced)
Main Computation (Parallelizable)
Total Time = T_sequential
Total Time = T_parallel < T_sequential

3. Fundamental Laws

Amdahl’s vs Gustafson’s

Amdahl's Law: Fixed Problem Size

Formula

```
Speedup = 1 / (S + (P / N))
```

Where:

- `S` : Sequential fraction
- `P` : Parallel fraction (1-S)
- `N` : Number of processors

Key Insight

Even with infinite processors, speedup is limited by sequential portion

Example: 10% sequential → Max speedup = 10×

Example Calculation

Given: 25% sequential, 75% parallel

With 4 processors:

```
S = 0.25, P = 0.75, N = 4
Speedup = 1 / (0.25 + 0.75/4)
        = 1 / (0.25 + 0.1875)
        = 1 / 0.4375
        ≈ 2.29×
```

With 16 processors: ≈ 3.37×

Maximum possible: 4×

Gustafson's Law: Scalable Problem Size

Formula

```
Speedup = S + P × N
```

Where:

- `S` : Sequential fraction (*after* parallelization)
- `P` : Parallel fraction (*after* parallelization)
- `N` : Number of processors

Key Insight

Problem size can scale with available resources Linear speedup is achievable!

Example Calculation

Given: 10% sequential, 90% parallel (*after* parallelization)

With 4 processors:

```
S = 0.10, P = 0.90, N = 4
Speedup = 0.10 + 0.90 × 4
        = 0.10 + 3.6
        = 3.7×
```

With 16 processors:

```
Speedup = 0.10 + 0.90 × 16
        = 0.10 + 14.4
        = 14.5×
```

Laws Comparison

Feature	Amdahl's Law	Gustafson's Law
Problem Size	Fixed	Scalable
Focus	Sequential limitations	Scaling potential
Speedup Limit	Bounded by 1/S	Theoretically unbounded
Perspective	Pessimistic	Optimistic
S & P Definition	Original execution	Parallel execution
Real-world	Legacy code optimization	Large-scale system design

When to Use Which?

4. Overview of

Parallel and Distributed Systems

for High Performance Computing

Parallel and Distributed Systems Overview

- Parallel and distributed systems are the backbone of modern HPC,
- enabling the execution of complex tasks using multiple processing units simultaneously.
- They can be categorized based on their Memory Architecture and Processing capabilities,

Note: ⚑
before we dive into specific architectures and programming models,
we need to understand the Flynn's taxonomy.

4.1. Flynn’s Taxonomy

Classifying Parallel Architectures

Flynn's Taxonomy Overview

Historical Context

- Introduced: Michael J. Flynn (1966, expanded 1972)
- Purpose: Classify computer architectures
- Basis: Number of concurrent instruction and data streams
- Modern Relevance: Still widely used today

Classification Parameters

Two fundamental parameters:

1. Single/Multiple Data Streams
2. Single/Multiple Instruction Streams

Four main categories:

- SISD, SIMD, MISD, MIMD

Note:

Flynn's taxonomy provides a valuable framework for understanding different types of computer architectures and remains fundamental for classifying parallel processing systems.

Flynn's Four Categories

Single Data Stream

SISD (Single Instruction, Single Data)

- Single instruction stream processed
- Sequential computer
- Historical baseline

MISD (Multiple Instruction, Single Data)

- Multiple instruction streams on single data
- Rarely used in practice
- Theoretical/fault-tolerant systems

Multiple Data Streams

SIMD (Single Instruction, Multiple Data)

- Same instruction on multiple data simultaneously
- Signal processing, GPU computing
- Vector operations, multimedia extensions

MIMD (Multiple Instruction, Multiple Data)

- Multiple instruction streams on multiple data
- Most widely adopted paradigm
- Modern parallel computing standard

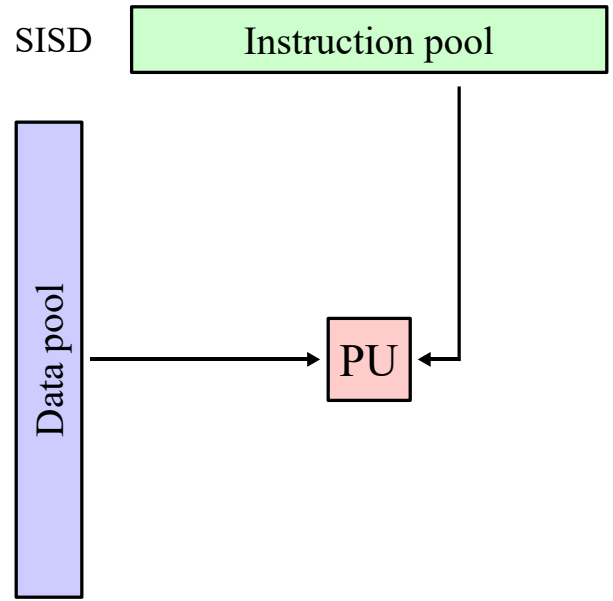
SISD: Single Instruction, Single Data

Characteristics

- Single processor executes one instruction at a time
- Single data stream processed sequentially
- Traditional sequential processing model
- Found in early computers and single-core processors

◦ Examples

- Original IBM PC
- Early personal computers
- Modern single-core processors



Note: We will not focus on SISD in this course, as it serves as a historical baseline for completeness.

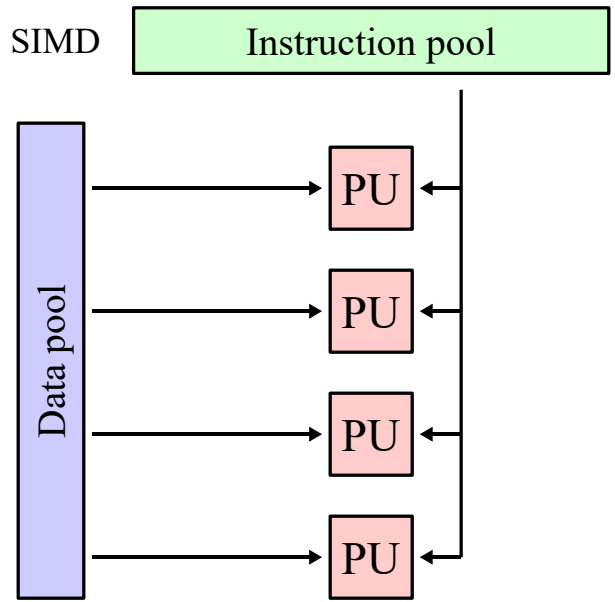
SIMD: Single Instruction, Multiple Data

Key Characteristics

- Same instruction applied to multiple data streams
- Parallel data processing
- Instructions executed sequentially or concurrently
- Essential for vectorization and GPU programming

◦ Modern Examples

- GPUs (Graphics Processing Units)
- Vector operations (SSE, AVX)
- Multimedia extensions in CPUs
- Signal processing applications



Course Importance: Crucial for understanding GPU programming (Chapter 3), Vectorization (Chapter 4), and OpenMP SIMD directives.

SIMD Subcategories (Flynn's 1972)

Flynn introduced three SIMD subcategories in 1972:

1. Array Processor (now SIMT)

- Each processing unit has separate memory and registers
- No shared memory between processing units
- Modern GPUs use this approach with SWAR ALUs

2. Pipelined Processor

- Read data from central resource (memory/register file)
- Each unit processes data fragments
- Write results back to central resource

3. Associative Processor

- Broadcast instruction plus register value
- Each unit makes independent decision
- Compare local data against broadcast value

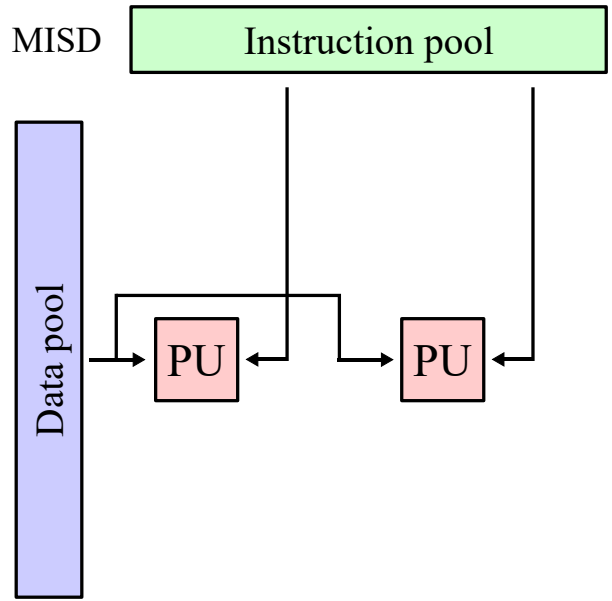
MISD: Multiple Instruction, Single Data

Characteristics

- Multiple instruction streams on single data
- Different processors perform different operations
- Used for redundancy or error correction
- Rare in practical implementations

◦ Examples

- Space shuttle flight control systems (theoretical)
- Fault-tolerant systems (hypothetical)
- Redundant processing systems
- Error correction applications



Note: Infrequently applied in practical contexts, mostly theoretical. Not essential for this course.

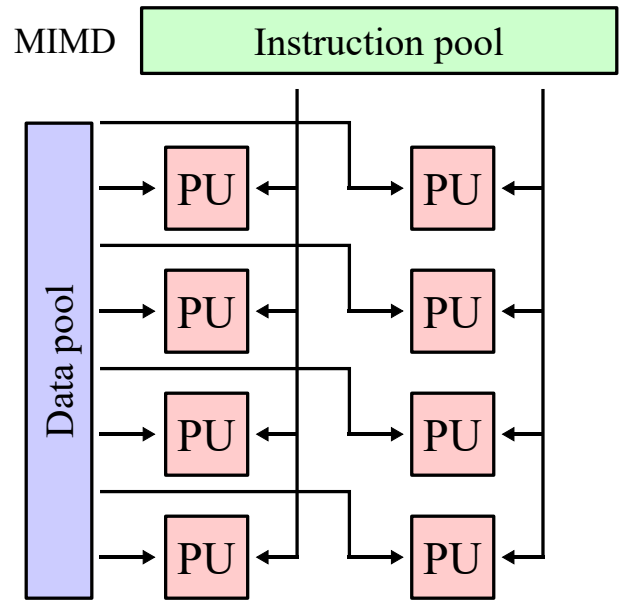
MIMD: Multiple Instruction, Multiple Data

Key Features

- Multiple processors executing different instructions
- Multiple data streams processed independently
- Most flexible and widely used architecture
- Independent operation with communication capability

Modern Examples

- Multi-core processors
- Computer clusters
- Supercomputers
- Distributed systems



Course Importance: Covers multi-core CPUs, OpenMP threading, and HPC cluster operations.

4.2. High Performance Computing Architectures

Implementation Categories

HPC System Categories

High Performance Computing (HPC) relies on three main architectural categories, each designed to address different computational challenges and scalability requirements.

- 1. Parallel Systems (Shared-Memory Architectures)
- 2. Distributed Systems (Distributed-Memory Architectures)
- 3. Accelerated Systems (Heterogeneous Architectures)

Common Goals

All three categories share common goals: reduce execution time, increase throughput, and tackle problems too large for a single processor.

1. Parallel Systems (Shared-Memory)

Core Concept

- Multiple processing units working together
- Tightly coupled architecture
- Common address space shared by all
- Efficient thread communication through shared variables

Programming Models

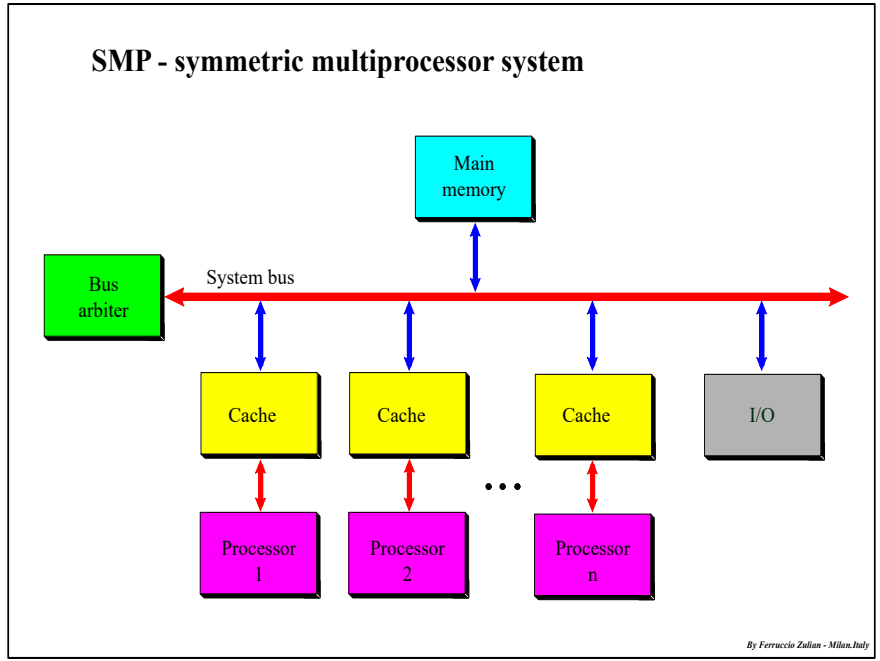
- OpenMP: Directive-based parallelization
- PThreads: API-based thread management
- Challenges: False sharing, NUMA locality

a) SMP: Symmetric Multi-Processing

Key Features

- **Single memory space** shared by all processors
- **Equal access** to all memory locations
- **Independent thread execution** per processor
- **Communication** through shared variables

- Examples
  - Multi-core processors (desktops/laptops)
  - Servers with multiple CPUs
  - High-end workstations
  - Some supercomputers



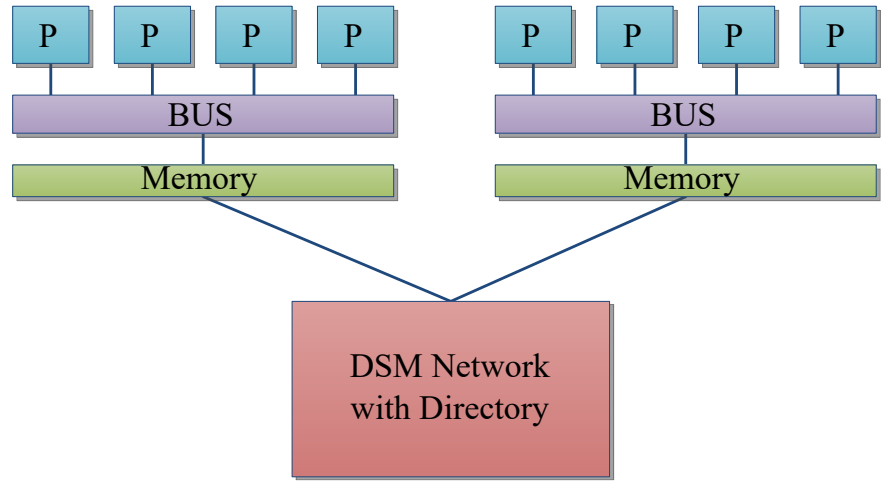
Course Importance: Chapter 2 (multi-core programming with OpenMP and PThreads).

b) NUMA: Non-Uniform Memory Access

Architecture Features

- Memory divided into multiple regions
- Processor-associated memory sections
- All processors can access all memory
- Speed difference: Local vs remote access

- Examples
  - High-end servers with multiple CPU sockets
  - Large-scale supercomputers
  - Multi-core processors with NUMA support



Course Importance:: Chapter 5 (optimization techniques for shared-memory systems).

2. Distributed Systems (Distributed-Memory)

Core Concept

- Geographically/logically separated resources
- Independent nodes with local memory
- Separate address spaces per node
- Network communication for collaboration

Programming Model

- MPI: Message Passing Interface
- Explicit communication required
- Data distribution as first-class concern
- Communication patterns critical for design

Distributed System Types

Cloud Computing

- Virtualized resources from service providers
- Scalability & flexibility on-demand
- Examples: AWS, Azure, Google Cloud

Grid Computing

- Geographically dispersed resources
- Internet-connected collaboration
- Example: WLCG for particle physics

Clusters

- Interconnected computers as single system
- Homo/Heterogeneous nodes
- Scientific computing and data centers

HPC Clusters

- High-speed interconnects (InfiniBand)
- Optimized software stacks
- Scientific research focus

Supercomputers

- Most powerful distributed systems
- Complex simulations capability
- Examples:Fugaku, Summit

Containerized Environments

- Lightweight, portable packaging
- Docker & Kubernetes orchestration
- Consistent deployment across environments

3. Accelerated Systems (Heterogeneous)

Core Concept

- Specialized hardware for specific workloads
- CPU + Accelerator combinations
- Distinct memory spaces require management
- Offload models for computation

Programming Models

- CUDA: NVIDIA-specific GPU programming
- OpenCL: Portable accelerator programming
- HIP/SYCL: Vendor-neutral approaches
- Explicit data movement management required

Accelerated System Types

CPU-GPU Hybrid

Architecture:
Heterogeneous computing
Memory: Unified/shared memory

- CPUs: Control flow, sequential tasks
- GPUs: Massively parallel computations
- Apps: Graphics, ML/DL, simulations

TPUs

Architecture:
Specialized for ML workloads
Memory: High-bandwidth memory

- TPUs: Google-designed accelerators
- Google-designed accelerators
- Neural network optimization
- High-throughput matrix ops

Domain-Specific

Architecture:
Tailored for specific tasks
Memory: Varies by application

- DSPs: Signal processing
- AI inference chips
- Cryptography accelerators

5. Programming Models and Interfaces

Programming Model Overview

Shared Memory

- **OpenMP**
  - Pragma-based hints for compilers
  - Parallelization directives
  - Fork-join model
- **PThreads**
  - Lower-level thread primitives
  - Explicit thread management

Distributed Memory

- **MPI**
  - Message Passing Interface
  - Explicit communication
  - Rich collective operations
  - Communicator abstractions
  - Industry standard

Accelerated Systems

- **CUDA**
  - NVIDIA-specific GPU model
  - Vendor-optimized performance
- **OpenCL**
  - Portable across vendors
  - Cross-platform compatibility
- **HIP/SYCL**
  - Vendor-neutral approaches

Programming Model Selection Criteria

Architecture Considerations

- Problem structure analysis
- Target platform capabilities
- Memory model requirements
- Communication patterns

Shared Memory: Fine-grained parallelism, tight coupling

Application Requirements

- Problem size and scalability needs
- Data movement overhead
- Development complexity tolerance
- Performance vs portability trade-offs

Distributed Memory: Large-scale problems exceeding single node capacity

Accelerated Systems: Require careful management of data movement but offer significant performance gains for suitable workloads

6. Conclusion

Summary and Next Steps

Key Takeaways

Architectural Understanding

- Flynn's Taxonomy provides classification framework
- SIMD crucial for GPU/vectorization programming
- MIMD dominates modern parallel computing
- Three HPC categories address different challenges

Practical Implications

- Programming model choice depends on architecture
- Shared memory: Easier for fine-grained parallelism
- Distributed memory: Better for large-scale problems
- Accelerated systems: High performance with complexity

Course Direction

The next chapters will delve deeper into these programming models, exploring their syntax, semantics, and best practices for effective parallel programming.